HYDER MEDIA · PLAYBOOK

The Modern AI-Discoverable SaaS Launch Playbook

Every optimization, defensive pattern, and launch tactic for shipping a SaaS that ranks across Google AI Overview, Perplexity, ChatGPT, and Claude.

Kenny Hyder · Hyder Media

MAY 2026 · V1.0

Every optimization, defensive engineering pattern, and launch tactic Hyder Media uses on every web product we ship — distilled into a vendor-agnostic playbook for any builder.

— Kenny Hyder, Hyder Media

Foreword: Why this playbook exists

Building for the web in 2026 is fundamentally different from building for the web in 2022.

In 2022, "discoverability" meant one thing: Google. You wrote keyword-targeted content, you earned backlinks, you waited for rankings to compound. Bing, Yandex, DuckDuckGo existed but were rounding errors.

That world is gone. The traffic you used to get from a #1 Google ranking now splits across at least seven distinct surfaces:

1. **Google web search** — still ~60% of organic traffic, but increasingly stripped to a snippet by Google AI Overviews
2. **Google AI Overviews** — pulls from a different stack than blue-link search; lives or dies by structured data
3. **Perplexity** — grounded answer engine; cites sources visibly; trained on the public web with heavy crawl
4. **ChatGPT search** — same family of behaviors as Perplexity, different surfacing logic
5. **Claude (Anthropic) web tools** — grounding from documents the user attaches, real-time fetches, and the model's training corpus
6. **Bing/Copilot** — small organic share, but where IndexNow actually lives + where Bing AI grounds
7. **Vertical AI tools** — every domain-specific AI (code agents, research bots, finance tools, etc.) is increasingly using OpenAI tool-calling against APIs you might not even know about

Each of these surfaces consumes a different signal. Google AI Overview wants `FAQPage` JSON-LD. Perplexity wants clean canonical text and Wikidata grounding. Claude's tool calling wants a polished OpenAPI 3.1 spec. Bing wants IndexNow pings.

The new mandate: **build for every surface simultaneously, with one set of files**. That's what this playbook is.

It's organized in five layers (plus a pre-flight chapter), built from foundation up. Skip nothing — they compound.

---

§0. Pre-flight: accounts and verifications

Before you can apply Layers 1-5, the foundation accounts have to exist and be verified. This is the boring, one-time setup that most playbooks skip — but you can't build entity grounding on top of an unverified domain.

If you've already built a few products, you can skim this and confirm everything's in place. If you're shipping your first one, do these in order before touching anything in §1.

0.1 Domain registration

- **Registrar matters less than you think.** Cloudflare Registrar, Namecheap, Porkbun are all fine. Cloudflare Registrar charges wholesale (~$8-10/yr for `.com`) with no markup and includes WHOIS privacy. Pick that unless you have a reason not to.
- **Use a `.com` if available.** Not for SEO — search engines don't care — but because humans still mentally complete `.com` first when they hear a brand. Avoid `.io` for consumer products (associated with crypto/dev tools); fine for B2B/dev.
- **Auto-renew + email forwarding.** Set auto-renew immediately. Forward `hello@`, `support@`, `legal@`, `security@`, `dmarc@` to your real email so you receive abuse reports + transactional sender feedback.

0.2 DNS — use Cloudflare even if you're not on Cloudflare for hosting

Move DNS to Cloudflare even if your domain is registered elsewhere and your site is hosted on Vercel/Netlify/etc. Reasons:

- Cloudflare's DNS propagation is the fastest in the industry (~30s vs hours for some registrars)
- Free DNS-level DDoS mitigation
- Cloudflare Email Routing (free) — lets you set up `support@yourdomain.com` → your real Gmail without paying for Google Workspace
- Page Rules / Transform Rules let you add cache headers without code changes
- Analytics that don't require JavaScript on the page

Configure these records day 1:

```
A      @                → [your host IP, e.g. Vercel's 76.76.21.21]
CNAME  www              → cname.vercel-dns.com  (or your host's CNAME target)
TXT    @                → "v=spf1 include:_spf.google.com include:resend.io ~all"  (see 0.7)
TXT    _dmarc           → "v=DMARC1; p=quarantine; rua=mailto:dmarc@yourdomain.com"
```

0.3 Hosting

For most SaaS products today, the right choice is **Vercel** (Next.js-friendly, generous free tier, fast cold starts) or **Cloudflare Pages** (cheaper at scale, slightly more setup). Netlify, Railway, and Render are also fine.

What this playbook assumes:

- Your host gives you serverless functions for API routes
- It auto-deploys from GitHub on push to `main`
- Custom domains land within seconds, not hours
- Environment variables can be set via CLI or dashboard

If your host doesn't do these, the patterns still apply but the exact commands differ.

0.4 Database + Auth

Supabase is the default for new SaaS in 2026: Postgres + Auth + Storage + Realtime in one product, generous free tier, RLS-first. **Neon** + **Clerk** is a common alternative (Neon for Postgres, Clerk for Auth). **Convex** if you want reactive queries instead of REST.

Pick one early — switching halfway through is painful because schema + auth assumptions ripple through your code.

Whatever you pick:

- **Enable RLS** (Row Level Security) on every table the moment you create it. Default-deny.

- **Have two clients**: an anon-key client for browser, a service-role client for serverless functions. Never expose the service-role key to the browser.
- **Use a managed Postgres connection pooler** if you're on serverless. Supabase ships one (`aws-0-us-west-2.pooler.supabase.com` on port 6543). Without it, cold-starting functions exhaust your connection limit fast.

0.5 Payments — Stripe setup

- **Create a Stripe account immediately**, even if you're not charging yet. The "live mode" account takes a few days for some verifications to clear. Don't wait until launch week.
- **Activate Stripe Customer Portal** at `https://dashboard.stripe.com/settings/billing/portal` BEFORE you ship the "Manage subscription" button. Without this configured, the portal API throws errors and the button silently fails. Check the boxes for: cancel subscription, update payment method, view invoices, download receipts. Save in both Test and Live mode separately.
- **Create products + prices in code** via a setup script, not in the dashboard. The script is reusable across Test/Live mode and survives the inevitable "let me delete this old price" cleanup. Keep your Price IDs in env vars, never hardcoded.
- **Set up webhooks** with explicit event lists. Don't subscribe to "all events" — narrow to the ones you actually handle (`checkout.session.completed`, `customer.subscription.{created,updated,deleted}`, `invoice.payment_{succeeded,failed}`).

0.6 Analytics — Google Analytics 4

- **Create a GA4 property** at https://analytics.google.com → Admin → Create Property. Name it your product name; timezone matters (set to your reporting timezone, not the data timezone). Currency in USD unless you have a strong reason otherwise.
- **Add a Web data stream**. Copy the Measurement ID (looks like `G-XXXXXXXXXX`) — you'll reference this in code.
- **Enable Enhanced Measurement** (the toggle is in the stream settings). This gives you pageviews, scroll depth, outbound clicks, file downloads, and form interactions automatically — saves you from manually instrumenting basic events.
- **Disable Google Signals** if you care about strict privacy/cookie compliance. Enable if you want demographic data and don't mind the consent banner implications.
- **Set up Conversions early.** Custom events fire as regular events by default; only events marked as "key events" appear in Conversion reports and can be imported into Google Ads. Mark `sign_up`, `purchase`, `begin_checkout` immediately (they auto-register after first fire, then you toggle them).

0.7 Email sender domain + SPF/DKIM/DMARC

This is the single most-skipped step in modern SaaS launches. If your transactional emails aren't authenticated, they go to spam — which means password resets, magic links, receipts, and alerts all fail silently. Users blame your product. You can't debug it without tooling that shows you the spam-folder verdict.

Use Resend (cleanest API, free up to 3,000 emails/mo) or **Postmark** (best deliverability, higher cost). Avoid SendGrid for new accounts — their shared IPs are reputation-damaged.

Setup steps:

1. In Resend (or your provider), add your sending domain (e.g. `yourdomain.com`).
2. Resend shows you DNS records to add — copy them all into your DNS:

3. **SPF** (TXT record on root): `v=spf1 include:resend.io ~all`. If you already have Google Workspace, combine: `v=spf1 include:_spf.google.com include:resend.io ~all`. You can only have ONE SPF record.
4. **DKIM** (TXT record on a specific subdomain like `resend._domainkey`): the long string Resend gives you. Copy verbatim.
5. **MX records** for return-path / bounce handling: Resend provides; usually a Resend-specific subdomain.
6. **DMARC** (TXT record on `_dmarc`): start at `p=quarantine` (suspicious mail goes to spam, not deleted). After 30 days with no false-positives, escalate to `p=reject`. Include `rua=mailto:dmarc@yourdomain.com` to get aggregate reports.
7. **Wait up to 48h for DNS propagation** (usually minutes with Cloudflare, hours with other registrars).
8. **Verify with mail-tester.com**: send a test email from your app to the address it provides → score 10/10. Anything less than 9/10 means a header is wrong; fix before launching.

This setup also protects you against email spoofing (someone sending phishing emails pretending to be you). DMARC + DKIM make that very hard.

0.8 Google Search Console

GSC is how Google tells you what queries you're ranking for, what URLs are crawl-errored, and what AI Overviews / featured snippets they've pulled from your site. Free, no excuse not to set up.

1. Go to https://search.google.com/search-console and click **Add property**.
2. Choose **Domain property** (not URL prefix) — covers all subdomains and protocols.
3. Verify via DNS TXT record. Cloudflare-hosted DNS = paste their TXT record, verify in 30 seconds.
4. Once verified:
5. **Submit your sitemap** at Settings → Sitemaps → `https://yourdomain.com/sitemap.xml`
6. **Set the preferred domain** (with `www.` or without — pick one, redirect the other)
7. **Configure international targeting** if you're targeting a specific country
8. **Add a property for each subdomain** you care about (e.g. `docs.yourdomain.com`) — domain-property covers them but sub-property gives you separate reports
9. **Connect to Google Analytics** at GA4 Admin → Search Console links → Link. Lets you see search query attribution in GA4.

0.9 Bing Webmaster Tools

Smaller traffic but where IndexNow + Bing AI grounding originate.

1. Sign in at https://www.bing.com/webmasters with the same Google account as GSC.
2. Click **Import from Google Search Console** — pulls your verified properties + sitemaps automatically. Saves 20 minutes of manual setup.
3. Confirm verification.
4. **Generate your IndexNow API key** under Settings → IndexNow. Save it; you'll deploy it at `https://yourdomain.com/[key].txt` and reference it on every IndexNow ping.

0.10 Stack diagram before you write a single line of code

Before opening the editor, draw the stack on paper:

```
Domain (registrar) → DNS (Cloudflare) → Host (Vercel)
                                          |
                  ┌──────────────────────┼──────────────────────┐
                  |                      |                      |
        DB (Supabase)              Payment (Stripe)        Email (Resend)
        Postgres + Auth            Subscriptions            Transactional
              |
        RLS policies on every table
        Service-role key for cron, anon key for browser

         Analytics:  GA4 (web + conversion events)
         Search:     GSC + Bing WMT
         AI grounding: Wikidata + llms.txt + JSON-LD + OpenAPI
         Datasets:   Hugging Face (if you publish open data)
```

Knowing the dependency graph prevents "wait, where does Stripe send the webhook again?" debugging mid-build.

0.11 Pre-flight verification checklist

Before you write a single feature, all of these must pass:

- [ ] Domain registered, auto-renew on
- [ ] DNS on Cloudflare (or equivalent fast DNS)
- [ ] Email forwarding for `hello@`, `support@`, `dmarc@` works (test it)
- [ ] Hosting account created, GitHub repo connected, hello-world page deploys on push
- [ ] Supabase (or alt) project created, two clients (anon + service-role) tested
- [ ] Stripe account created, Customer Portal configured, test webhook event fires
- [ ] Resend domain verified, mail-tester.com score 10/10
- [ ] DMARC report email arrives within 24h
- [ ] GA4 property created, Measurement ID copied
- [ ] Search Console property verified, sitemap accepted
- [ ] Bing Webmaster Tools imported, IndexNow key generated and deployed

This checklist will save you a week of mid-build interruptions. Get it done in one sitting.

---

§1. The 5-layer model (TLDR)

```
┌──────────────────────────────────────────────────────┐
| Layer 5: Distribution                                |
|  Press, social, journalist outreach, directory listings |
├──────────────────────────────────────────────────────┤
| Layer 4: Conversion + analytics                      |
|  GA4 events, success URLs, error handling            |
├──────────────────────────────────────────────────────┤
| Layer 3: Compliance + headers                        |
|  CSP, HSTS, COOP, Permissions-Policy, RLS, secrets   |
├──────────────────────────────────────────────────────┤
| Layer 2: Search + answer engines                     |
|  JSON-LD, OpenAPI, sitemaps, IndexNow, FAQs          |
├──────────────────────────────────────────────────────┤
| Layer 1: Entity identity                             |
|  Wikidata, llms.txt, canonical IDs, structured grounding |
└──────────────────────────────────────────────────────┘
```

Each layer requires the one below it. You can't get good GA4 conversion attribution if your URLs aren't canonical (Layer 2). You can't get into Google AI Overviews if your headers leak mixed content (Layer 3). You can't pitch journalists effectively if your Wikidata entry is broken (Layer 1).

Build bottom-up. Reading top-down (skipping foundations) is the most common reason launches don't compound.

---

§2. The "everything is one set of files" principle

If you take one architectural idea from this playbook, take this one:

Don't write SEO content, then separately write LLM-grounding content, then separately write API docs. Write each piece of canonical information *once*, in a structured format, and project it into every surface that needs it.

Concrete example — your product's tier metadata lives in `lib/tiers.ts` (TypeScript) and is consumed by:

- The pricing page UI (rendering)
- The Stripe checkout flow (price ID lookup)
- The JSON-LD `WebApplication` `offers` block (SEO + AI)
- The OpenAPI spec's response examples (developer docs)
- The GA4 `purchase` event's `item_id` and `value` (analytics)
- The `llms.txt` pricing block (AI grounding)
- The press release boilerplate (distribution)

When any one of these changes, you change it in one place. Drift between surfaces is what causes LLMs to lose confidence in your entity ("which is the canonical price — the one in the pricing page or the one in the API docs?") and downrank you as a source.

Same pattern applies to your Wikidata Q-ID, your canonical product name, your pricing tiers, and any other "the source of truth" data:

- Q-ID lives in your config → referenced by JSON-LD `sameAs`, llms.txt, OpenAPI `x-wikidata` extension, citation blocks, press release
- Canonical product name in your config → rendered into every meta title, JSON-LD `name`, OG image, llms.txt header

The corollary: don't put canonical facts in CMS-rendered marketing pages. Put them in code/config, render the marketing pages from that, and project the same facts into the AI-readable formats.

---

§3. Layer 1 — Entity Identity

The deepest signal that determines whether an AI tool will mention you. Three artifacts:

3.1 Wikidata entry

A Wikidata Q-item is the canonical anchor that lets multilingual LLMs disambiguate you from everything else with a similar name. It's also what populates Google's Knowledge Graph (the right-rail card on search results pages).

Create the entity early. The bar is low — verifiable existence (a website, a press mention, or being listed in another database is sufficient). Cost: free.

Required claims for a SaaS product:

Property	Use	Notes
`P31` (instance of)	What kind of thing this is	At minimum `Q35127` (website) + `Q1668024` (web application). Avoid using "date of establishment" (`Q3406134`) — that's not a class.
`P17` (country)	Where you're based	E.g. `Q30` for United States
`P856` (official website)	Your URL	One claim
`P571` (inception)	Launch date	Use full date precision (precision 11)
`P112` (founded by)	Founder Q-ID	Create a Q-ID for yourself first if you don't have one
`P127` (owned by)	Operating org Q-ID	Same
`P154` (logo image)	Commons filename of your logo	See §3.1.2 below
`P2002` (X/Twitter username)	Without the `@`	E.g. `yourbrand`
`P1813` (short name)	Your brand short form	With language tag, e.g. `en:"YourBrand"`
`P527` (has part)	Q-IDs of upstream services or data sources	Anchors the entity graph
`P921` (main subject)	What the product is about	Often the same Q-IDs as P527
`P452` (industry)	Industry Q-ID	Search Wikidata for your industry to find its Q-ID

Avoid these mistakes:

- Don't use `P275` (copyright license) unless your *entire site content* is CC-licensed. SaaS products usually aren't.
- Don't put marketing copy in descriptions. Wikidata descriptions are factual, single-sentence, and translatable.

Multilingual labels. Add labels in 10+ languages (the brand name in each — usually the English form works for tech products). Multilingual LLMs ground better when the entity is recognized in their training language.

Aliases. Add every search-query variation users might type. These are what query expansion uses internally.

3.1.1 Editing at scale: QuickStatements vs bot password vs UI

For new accounts (under 4 days old + under 50 edits), Wikidata's "autoconfirmed" gate blocks QuickStatements. Three paths:

1. **Wait for autoconfirmed** — 4 days + 50 manual edits. Annoying but unblocks everything.
2. **Bot password** — Create a scoped bot password at `Special:BotPasswords` with only "Edit existing pages" granted. Write a script that uses the MediaWiki API. Bot passwords bypass

autoconfirmed for `wbeditentity` on existing items.

3. **Manual UI edits** — Slow but always works for existing items. Use for handful-of-edits cases.

Defensive note on bot passwords: they're tied to your account but separate from your main password. Revoke immediately after the batch run completes.

3.1.2 Logo on Wikimedia Commons

For Google Knowledge Panel, you need `P154` populated. That requires the logo file to be hosted on Wikimedia Commons (not your own site).

Upload via `Special:UploadWizard`. The wizard will warn you about logos uploaded as "Own work" with CC licenses because that combination is the common-vandalism pattern. If you are genuinely the copyright holder, you can proceed past the warning. Best practices:

- In the file description, disclose constituent parts honestly. ("Composite work — icon adapted from [library] (ISC license), background and color original.")
- Choose `CC-BY-SA-4.0` for "own work" cases, or `PD-textlogo` / `PD-shape` for logos that are clearly below the originality threshold for copyright (basic shapes + text).

3.1.3 Deprecating wrong claims when you can't delete them

New accounts can't delete claims due to abuse-filter rules. **Mark them as deprecated** instead — set `rank: deprecated` and add a `P2241` qualifier with `Q1193907` ("incorrectly entered claim"). All data consumers ignore deprecated claims. This is the standard Wikidata pattern.

3.2 llms.txt

`/llms.txt` at the root of your domain is the canonical machine-readable narrative of your site for LLMs. Anthropic's Claude, Perplexity, and most AI crawlers fetch this and feed it into their understanding of your site.

Structure:

1. **Lede paragraph** — one-line product description, your founding date, your founder.
2. **Canonical identifiers** — Wikidata Q-ID, X handle, GitHub, Hugging Face, etc. This is the most important block. List every identifier so AI tools can ground cross-source.
3. **Disambiguation** — "This is NOT X (Q-id)" — explicit negative anchoring for entities that share names with adjacent things.
4. **Core concepts** — links to your `/learn` glossary pages.
5. **Live data + sections** — links to dynamic pages.
6. **API docs** — link to OpenAPI spec + free demo key + endpoints list.
7. **Public datasets** — Hugging Face, direct CSV, license.
8. **Methodology** — bullet-pointed canonical math/algorithms (not opinion).
9. **Citable facts** — single-sentence factual claims with sources where applicable.
10. **Citation block** — APA + BibTeX format. LLMs cite you using these formats when they ground.

Avoid:

- Marketing fluff. Keep the voice neutral and factual.
- Stale links. Update llms.txt when you change pricing, drop features, or migrate URLs.
- Pricing as a "from $X" — list the full price points so LLMs can answer "how much does it cost" without scraping the pricing page.

See `/templates/llms.txt` for a fill-in-the-blanks version.

3.3 Canonical JSON-LD on every page

This is the structured data Google + Bing + AI Overviews parse from your HTML to populate Knowledge Panels, FAQ snippets, pricing carousels, etc.

Mount four schemas on every page:

1. **`Organization`** — your company / founder. Include `sameAs` array pointing at Wikidata, GitHub, HF, X.
2. **`WebSite`** — your domain. Include a `SearchAction` pointing at your site search URL so Google's sitelinks search box can render.
3. **`WebApplication`** — your product. Most important schema for SaaS. Include:
4. `applicationCategory` (e.g. `BusinessApplication`, `FinanceApplication`, `UtilitiesApplication`)
5. `operatingSystem: "Web"`
6. Full `offers` array with each pricing tier's `price`, `priceCurrency`, `priceSpecification.unitText` (`MON` / `ANN`)
7. `sameAs` array (same as Organization, repeated — schemas don't inherit)
8. `about[]` array referencing related Wikidata Q-IDs so crawlers traverse the graph
9. `audience.audienceType` description
10. **`FAQPage`** — per-page Q&A. Generate dynamically from real data on the page. Each Q&A becomes a Google AI Overview answer candidate.

Mounting pattern (Next.js example):

```
// app/layout.tsx — sitewide schemas
<head>
  <JsonLd data={[organizationLd(), websiteLd(), webApplicationLd()]} />
</head>

// app/[some-page]/page.tsx — per-page schemas
<JsonLd data={[breadcrumbLd(items), itemListLd(name, items), faqLd(qaItems)]} />
```

Always emit JSON-LD inline via `<script type="application/ld+json">`. Never via external URL — crawlers don't follow.

See `/templates/webApplicationLd.ts` for a typed TypeScript generator.

---

§4. Layer 2 — Search & Answer Engines

4.1 Per-page metadata

Every page should export `generateMetadata` (Next.js App Router) or equivalent returning:

- `title` (≤60 chars, includes brand)
- `description` (≤155 chars, action-oriented)
- `alternates.canonical` (the canonical URL, including site URL prefix)
- `openGraph` block (title, description, url, type, images with explicit width/height, siteName)
- `twitter` block (card type, title, description, images)
- `keywords` array (still consumed by Bing despite Google ignoring it)

Open Graph images must be: - 1200×630 px (the 1.91:1 ratio) - PNG with non-empty body - Wrapped in object form with explicit `width`, `height`, `type`, `alt` properties (X / Twitter crawler bug: it surfaces "Card error" if you only provide URL)

Generate OG images dynamically via Next.js `next/og` `ImageResponse` (or equivalent). Wrap in try/catch with a brand-fallback card — `ImageResponse` can throw mid-render on edge cases (rare data shapes, unicode glyphs missing in fonts) and an empty 200 response will silently break previews on every social platform.

4.2 OpenAPI 3.1 spec (for LLM tool-calling, not just developer docs)

If your product has an API, ship a fully-fleshed OpenAPI spec — not because human developers need it (they often don't), but because LLM tool-calling frameworks (LangChain, LlamaIndex, OpenAI function calling, Anthropic tool use) all auto-generate clients from OpenAPI.

Required fields beyond the basics:

- `info.contact.email` + `info.contact.url`
- `info.termsOfService` URL
- `info.license` block
- `info.x-logo.url` (extension recognized by Stoplight, ReadMe, others)
- `info.x-wikidata` extension with your Q-ID (custom, but readable by any LLM)
- `externalDocs.url` pointing to your rendered docs page
- `tags[]` to group endpoints (LLMs use these to pick the right endpoint)
- `servers[]` with `description` (not just URL)

For each endpoint:

- `summary` (one line) and full `description`
- `operationId` (unique; LLMs use this as the tool function name)
- `tags` linking to the top-level tags
- Every parameter must have `description` and an `example`
- **Most-skipped**: full example response bodies in `content.[mime].example`. This is what LangChain and Claude tool use bind to when generating client code. Without examples, LLMs hallucinate response shapes.

4.3 Sitemap, robots, IndexNow

- **`/sitemap.xml`** — every public page. For high-volume sites, split into `sitemap-index.xml` + sub-sitemaps. Update on every cron tick that produces new content.
- **`/robots.txt`** — explicit `Allow: /` for AI crawler user agents (GPTBot, ClaudeBot, PerplexityBot, etc.) unless you intend to block them. Many sites block these by accident copying old robots templates.

Recommended robots.txt for AI-friendly indexing:

```
User-agent: *
Allow: /
Sitemap: https://yourdomain.com/sitemap.xml

# Explicitly allow AI crawlers (some sites block by default)
User-agent: GPTBot
Allow: /
User-agent: ClaudeBot
Allow: /
User-agent: PerplexityBot
Allow: /
User-agent: anthropic-ai
Allow: /
User-agent: Google-Extended
Allow: /
User-agent: CCBot
Allow: /
User-agent: Applebot-Extended
Allow: /
```

- **IndexNow** — Bing/Yandex/Naver use this protocol. After every batch of new URLs, POST to `https://api.indexnow.org/IndexNow` with your key (deployed at `/keyfile.txt` matching the key value). Google doesn't use IndexNow but discovers via sitemap re-crawl, so you still need the sitemap.

Bundle IndexNow into your cron ingestion. Every new URL gets submitted within minutes, not days.

4.4 Slug strategy

Canonical URLs are the foundation of SEO + cross-source citation. Three rules:

1. **Slugs must be unique per entity instance.** If your data model has events that repeat over time (sports games on multiple dates, recurring meetings, daily snapshots), include a date suffix in the slug. Otherwise `slug` collides, your DB lookup uses `maybeSingle()`, and the URL 404s.
2. **One canonical URL per resource.** Use a redirect chain from legacy / aliased URLs to the canonical.
3. **Use 307 (temporary), not 308 (permanent), for slug-redirects.** Permanent redirects get aggressively cached by browsers and CDNs. If you ever rename a slug (which happens — collision fixes, taxonomy changes), the cached 308 will keep sending users to the old URL until per-browser cache expires (could be days). 307 keeps redirects dynamic at the cost of slight SEO equity. Crawlers still see the canonical via the `<link rel="canonical">` tag in HTML, so you don't lose ranking.

4.5 FAQ generation from real data

Static FAQ pages are dead weight in the AI Overview era. Google AIO + Perplexity rank dynamic, page-specific FAQ schemas higher because they answer the user's intent for *that page*.

Generate `FAQPage` schema per page from real data your application already knows. Each Q&A pair becomes both visible HTML (renders in a `<details>` block on the page) AND `FAQPage` JSON-LD. The same data drives both, so they can't drift.

Build FAQ generators for every page type: category index, individual item, product detail, pricing, tools.

---

§5. Layer 3 — Compliance & Security

5.1 Content-Security-Policy (the one that breaks the most launches)

CSP is the header that breaks more launches than any other. Strict CSP is essential for protecting against XSS, but every third-party integration you add (Stripe, GA4, Supabase, fonts, etc.) adds to your allowlist.

The mistakes you'll make:

- **`form-action`** must include every domain a form might submit to *after redirects*. Browsers check `form-action` against the **final** destination, not the initial POST target. If your form posts to `/api/stripe/portal` which 303-redirects to `billing.stripe.com`, your CSP must include `https://billing.stripe.com` in `form-action`. This is a common bug — the portal button silently fails in browser console with `form-action 'self'` violation.
- **`script-src`** for `@next/third-parties/google` (GA4 loader) requires `https://www.googletagmanager.com` AND `https://*.google-analytics.com`. Both are needed; only one isn't sufficient.
- **`connect-src`** for Supabase real-time requires `wss://*.supabase.co` (WebSocket) in addition to `https://*.supabase.co`.
- **`frame-src`** for Stripe Checkout requires `https://js.stripe.com`, `https://checkout.stripe.com`, AND `https://hooks.stripe.com` if you embed any Stripe Elements.
- **`'unsafe-inline'` on `script-src`** is required by Next.js App Router's RSC payload bootstrap. Nonce-based CSP would be more secure but breaks Turbopack dev mode. Accept `'unsafe-inline'` for now; revisit when Next.js ships nonce support that works with Turbopack.

A reference CSP for SaaS with Stripe + GA4 + Supabase is in `/templates/csp-reference.md`.

5.2 The other security headers (set them all)

Header	Value
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains; preload`
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`DENY`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`camera=(), microphone=(), geolocation=(), interest-cohort=(), browsing-topics=()`
`Cross-Origin-Opener-Policy`	`same-origin-allow-popups`
`Cross-Origin-Resource-Policy`	`same-origin` (default) — override to `cross-origin` for OG images so social crawlers can fetch t
`X-DNS-Prefetch-Control`	`on`

After deployment, score yourself at https://securityheaders.com — aim for A+.

5.3 Database-level access control (RLS)

If you're on Supabase / Postgres: **enable Row Level Security on every table the moment you create it**. Default-deny. Then write explicit policies for each access pattern.

The most common mistake: shipping a table with RLS enabled but no policies. That blocks legitimate reads. Symptoms: app works for the service-role client (in serverless functions) but everything is empty for end users via the anon-key client.

Always have: - A read policy that gates by `auth.uid()` for user-owned rows - A write policy that gates the same way - A service-role bypass for cron jobs (the service role key bypasses RLS automatically — that's the design)

For analytics / billing tables, RLS keeps users from reading each other's billing history. For ingest tables (your scraped or public data), often you want public read with no write — `CREATE POLICY ... FOR SELECT USING (true)` works.

5.4 Environment variable hygiene (the trailing-newline bug)

If you set environment variables via CLI piped from `echo`, the trailing newline becomes part of the value. `vercel env add NAME prod` (and most other CLI tools) read stdin verbatim. So `echo "value" | vercel env add` stores `"value\n"`.

The Stripe SDK, when constructed with a key containing a trailing newline, puts that into the `Authorization` header. The embedded `\n` corrupts the HTTP header, requests never reach Stripe, the SDK retries twice, then throws "An error occurred with our connection to Stripe. Request was retried 2 times."

The error message *sounds* like a network problem. It's not. It's local header corruption from a stored newline.

Rules:

1. **Always use `printf %s "value" | <cli> env add`**, never `echo`. `printf %s` doesn't add a newline.
2. **In code, `.trim()` every env var read** before use:
   `ts const key = process.env.STRIPE_SECRET_KEY?.trim();` This is defense in depth. Even if you religiously use `printf`, the next developer might use `echo`. The trim costs nothing.
3. **For sensitive env vars (API keys, webhook secrets), add a length check**:
   `ts if (process.env.STRIPE_SECRET_KEY?.length !== 107) { console.warn("STRIPE_SECRET_KEY has unex`
   Stripe live keys are exactly 107 characters. Anything else is a sign the value is corrupted.

5.5 Webhook signature verification + error handling

Every webhook endpoint must:

1. **Verify signature** before parsing body. Stripe, Twilio, Resend all have signature verification helpers. Use them.
2. **Trim the webhook secret** before passing to the verifier (same newline bug applies here).
3. **Wrap everything in try/catch**. Webhook failures should surface useful logs, not generic 500s.
4. **Return 200 even for "we recognize this event but it's not interesting"** — non-200 responses cause webhook senders to retry, eventually hitting their max-retries limit and getting marked as broken on the sender side.

5.6 Secret-rotation hygiene

Rotate any secret that gets logged, displayed in an error message, or pasted in a chat / ticket. If you see a key in a screenshot, in someone's terminal history, or in a Slack message — rotate immediately.

Maintain a rotation log:

```
2026-MM-DD  Stripe webhook secret rotated (was visible in test webhook URL)
2026-MM-DD  Wikidata bot password revoked (one-time use complete)
2026-MM-DD  ...
```

---

§6. Layer 4 — Conversion + Analytics

6.1 GA4 events for SaaS (the events you actually need)

Pageviews come for free with `<GoogleAnalytics>`. Custom events you must fire:

- **`sign_up`** — on first /dashboard load after auth callback. Method: `magic_link`, `oauth`, `password`, etc.
- **`begin_checkout`** — when the user clicks Subscribe (before redirect to Stripe). Include items array with tier as `item_id` and price as `value`.
- **`purchase`** — on Stripe success redirect back to your site. Include `transaction_id` (Stripe subscription ID) for deduplication.

GA4 dedupes `purchase` events by `transaction_id`. If you fire the same `purchase` twice from a page refresh, GA4 collapses them. This is the right behavior.

6.2 The webhook race condition (purchase event timing)

The classic bug: user completes Stripe checkout → Stripe redirects browser back to `/dashboard?upgraded=1` → your `/dashboard` reads the user's tier from your DB → tier is still "free" because Stripe's webhook hasn't reached your `/api/stripe/webhook` yet → your `trackPurchase` function bails because "free" tier has $0 value.

The webhook does eventually update the tier, but ~200ms after the browser redirect. Too late.

Fix: include the tier in the Stripe success URL.

```
const successUrl = `${SITE_URL}/dashboard?upgraded=1&tier=${tier}`;
```

Then in your `ConversionTracker` client component, read the tier from the URL, not from props (which come from a server-rendered DB read):

```
const urlTier = params.get("tier") || propTier;
trackPurchase(urlTier, transactionId);
```

The URL is authoritative for the *intent* of the checkout. The DB will catch up via webhook for everything else (gating, billing display).

6.3 The gtag race condition (events silently dropped)

The other classic bug: `@next/third-parties/google` loads the GA4 script with `strategy="afterInteractive"`. Your `useEffect` in `ConversionTracker` runs *before* the script finishes loading. If you call `window.gtag(...)`, gtag is `undefined`, and your guard (`if (typeof window.gtag !== "function") return`) silently drops the event.

Fix: push to `window.dataLayer` directly, not via `window.gtag`. The GoogleAnalytics component initializes the `dataLayer` array synchronously (it's part of the inline stub). Push goes through regardless of whether the full GA4 script has loaded:

```
function gtag(action: string, params?: Record<string, unknown>) {
  if (typeof window === "undefined") return;
  window.dataLayer = window.dataLayer || [];
  window.dataLayer.push(["event", action, params]);
}
```

This is the same pattern GA4's own gtag() helper uses internally. We just skip the call indirection so a script-load race can't drop the event.

Verify by inspecting `window.dataLayer` in DevTools console after navigating to a conversion URL. You should see entries like `['event', 'purchase', {value: 10, ...}]`. If you don't, the event isn't firing.

6.4 Mark events as conversions in GA4 (or they don't count)

Custom events fire as regular events by default. They only show up in Conversions reports + are importable into Google Ads if you flip them to "key events":

```
GA4 → Admin → Events → toggle "Mark as key event" on each event
```

They take ~24-48h to propagate from the first fire before appearing in the Events list. Mark them on day 2 of your launch, not day 0.

6.5 Error handling that surfaces causes

Every serverless route that calls a third-party API must:

1. Wrap the call in try/catch
2. Log the full error + stack to runtime logs
3. Redirect the user to a meaningful URL with the error reason as a query param

Example pattern:

```
try {
  const session = await stripe.checkout.sessions.create({...});
  return NextResponse.redirect(session.url);
} catch (e) {
  const msg = e instanceof Error ? e.message : String(e);
  console.error("[checkout] Stripe create failed", { error: msg, stack: e?.stack });
  const reason = encodeURIComponent(msg.slice(0, 200));
  return NextResponse.redirect(`${SITE_URL}/pricing?error=checkout_failed&reason=${reason}`);
}
```

This pattern turns "this page isn't working" 500s into actionable URLs. When a user reports a problem, they paste the URL → you see the actual error message immediately. No log spelunking required.

---

§7. Layer 5 — Distribution

7.1 The ranked launch channels

Not all channels are equal. Ranked by ROI per minute spent:

1. **Wikidata + JSON-LD** (already shipped, day-0)
2. **Hugging Face dataset card** (if you have public data, day-0)
3. **GitHub public docs repo** (already shipped, day-0)
4. **Direct journalist pitches** (5 reporters, ~30 min, highest conversion to actual press)
5. **Your own X/LinkedIn/Bluesky** (3 min total)
6. **Show HN** (Tuesday 7-9 AM ET; 2 min submit + 10 min first-comment anchor)
7. **Reddit posts in 3 subs** (15 min, stagger across 3 days)
8. **Paid wire service** (one is enough — EIN Presswire $99 or PRWeb $99)
9. **Product Hunt** (schedule for next Tuesday, ~20 min for full submission)
10. **Free directories** — BetaList, AlternativeTo, SaaSHub, Crunchbase (~20 min total)

Wire services are *not* where readers come from — they're where SEO backlinks + AI-training-data crawl come from. Pitch journalists for actual coverage.

7.2 Platform-specific copy variants

Don't reuse the same copy across platforms. Each platform's algorithm and audience weights different things.

- **X thread (8 tweets)**: hook first, value middle, CTA last. Link only in final tweet (X suppresses tweets with links).
- **LinkedIn**: long-form (1300+ chars), no link in body, link in first comment. Personal profile beats company page.
- **Bluesky**: 5 posts, 300 chars each. Smaller audience but scraped by Claude/Perplexity.
- **Show HN**: title + URL + body. First comment within 10 min is critical — it anchors discussion and prevents "no comments" decay.
- **Reddit**: each sub gets a distinct framing. Cross-posted identical text gets removed by mods.

7.3 Journalist outreach

The highest-conversion channel by far.

Process:

1. Identify 5 reporters who've covered your space in the last 12 months. Use Muckrack, Twitter search, or direct googling.
2. Read one of each reporter's recent pieces. Note something specific to reference.
3. Email each reporter with a personalized pitch — reference the article, then give your angle.
4. **Don't BCC**. Each pitch is its own email. Reporters check, and they hate templates.
5. **Don't attach a PDF press release**. Paste the relevant 2-3 paragraphs inline.
6. Follow up at day 3 and day 7 with different angles each time. Stop after day 14.

Template in `/templates/journalist-pitch.md` . The key insight: lead with what's interesting to *their readers*, not what's interesting to you.

7.4 IndexNow on every launch URL

Before pitching journalists, fire IndexNow at every URL you want crawled:

```
curl -X POST "https://api.indexnow.org/IndexNow" \
  -H "Content-Type: application/json" \
  -d '{
    "host": "yoursite.com",
    "key": "your-key",
    "keyLocation": "https://yoursite.com/your-key.txt",
    "urlList": ["...all your important URLs..."]
  }'
```

Bing, Yandex, Naver pick it up within hours. Google doesn't but discovers via sitemap re-crawl.

---

§8. Defensive engineering patterns

The named bugs that have cost real launches real hours. Apply by symptom-matching:

8.1 The Stripe newline bug

Symptom: SDK calls fail with "Connection error, retried 2 times" or "no such price: 'price_XXX\n'". **Root cause:** Env var has trailing newline from `echo` piping. **Fix:** Use `printf %s` not `echo` when setting; `.trim()` every read in code.

8.2 The 308 cache-poisoning trap

Symptom: After changing a redirect target (slug fix, URL migration), users still land on the old URL even though server logic is correct. **Root cause:** `permanentRedirect()` (308) is aggressively cached by browsers + CDNs. **Fix:** Use `redirect()` (307, temporary) for redirects whose target might ever change. 308 only for truly-immutable URL migrations.

8.3 The gtag race condition

Symptom: Conversion events fire (according to your code) but never appear in GA4 Realtime. **Root cause:** `window.gtag` is undefined when your `useEffect` runs. **Fix:** Push to `window.dataLayer` directly. The dataLayer is initialized synchronously by the GA loader; the gtag function is hydrated later.

8.4 The webhook race condition

Symptom: `purchase` events fire with wrong value (often $0) because the tier in your DB hasn't updated yet. **Root cause:** Stripe redirects browser back faster than its webhook reaches your server. **Fix:** Pass authoritative state in the success URL (e.g. `&tier=pro`); read from URL, not DB, for the event payload.

8.5 The slug collision 404

Symptom: URLs return 404 even though the resource exists. **Root cause:** Multiple resources have the same slug (e.g., the same matchup repeats across days, daily snapshots, recurring meetings). `maybeSingle()` returns null when multiple rows match. **Fix:** Append a uniqueness suffix to the slug for repeating resource types (e.g., date for sports games, instance ID for recurring events).

8.6 The Turbopack RGBA favicon

Symptom: Next.js builds fail with "The PNG is not in RGBA format" on `app/favicon.ico`. **Root cause:** PNG optimization tools (oxipng with max settings) detect that your logo has no alpha variation and strip the alpha channel to save bytes. Turbopack's ICO parser strictly requires `color_type=6` (RGBA). **Fix:** Force RGBA in the PNG entries inside the ICO. Use Pillow's `optimize=True` only — don't run aggressive optimizers on `favicon.ico`'s contents.

8.7 The X dedup filter

Symptom: X (Twitter) starts marking your automated posts as "duplicates" even if they're not literally identical. **Root cause:** X has a fuzzy duplicate-detection filter that flags posts sharing too many tokens in the same order. **Fix:** 1. Use 10+ template variants for any automated content (rotate by `hash(id) % templates.length`). 2. Cap posts per cron run to 1 (consecutive posts are ≥10 minutes apart). 3. Dedup per resource for 24h via DB lookup, not just per-run Set.

8.8 The settled-market "phantom price" trap

Symptom: Aggregated pricing data shows 1% / 99% (or similar near-extreme) values on resolved markets, polluting category pages and edge alerts. **Root cause:** Settled exchange/market data sources keep broadcasting their final settlement prices forever. If you fall back to "last trade price" when bid/ask is unavailable, settled markets feed in. **Fix:** 1. Require valid bid AND ask with reasonable spread before using midpoint (e.g., `bid > 0`, `ask − bid <= 0.1`, `ask < 1.00`). 2. Archive resources whose source data has stopped advancing (`fetched_at` stale > 6h) even if no explicit `closed_at` is set.

8.9 The OG-cache-on-social trap

Symptom: Even after fixing OG image generation, X/Twitter/Facebook show old broken card previews on shared links. **Root cause:** Social platforms cache OG cards aggressively (sometimes for days). They don't re-fetch unless you tell them. **Fix:** Use platform-specific debuggers to bust the cache: - X: https://cards-dev.twitter.com/validator (deprecated but sometimes still works) - Facebook: https://developers.facebook.com/tools/debug/sharing/ — click "Scrape Again" - LinkedIn: https://www.linkedin.com/post-inspector/ For each platform, paste the URL and click their re-scrape button.

8.10 The CSP-form-action-on-redirect trap

Symptom: Form submissions silently fail with browser console error about `form-action`. **Root cause:** Browsers check CSP `form-action` against the **final** destination after redirects, not the initial POST target. **Fix:** Include every domain a form might redirect to in `form-action`. Stripe Customer Portal redirects to `billing.stripe.com`; checkout to `checkout.stripe.com`. Both need to be in your `form-action` directive.

8.11 The Vercel build → silent old-deploy trap

Symptom: Your `git push` "succeeded" but production still shows old behavior. You assume the deploy didn't happen. **Root cause:** Vercel **did** try to deploy, but the build failed. Vercel keeps serving the last "Ready" deployment when new builds fail. Without checking deploy status, you'll waste hours debugging code that isn't even live. **Fix:** After every push, check `vercel ls` output. Look at the latest deployment status: - `Ready` = live, your code is serving - `Building` = still building, wait - `Error` = build failed, your changes are NOT live. Read the build logs with `vercel inspect <url> --logs`.

This is the #1 "ghost bug" pattern. Always verify deploys.

8.12 The deliverability degradation

Symptom: Magic-link emails stop arriving for some users; gradually more users report it; eventually password resets fail for everyone. **Root cause:** Sender domain reputation has degraded — usually because SPF / DKIM / DMARC were set up wrong, or because a small number of users marked your emails as spam. **Fix:** 1. Verify SPF + DKIM + DMARC at mail-tester.com — must be 10/10. 2. Check Google Postmaster Tools (https://postmaster.google.com) — shows your domain's spam rate and reputation. 3. If reputation is low, slow your send volume to known-good recipients (your own team, paying customers) for 7-14 days. Reputation recovers gradually. 4. Never buy email lists. Never send marketing email from your transactional domain — use a subdomain (e.g. `marketing@news.yourdomain.com`).

---

§9. Checklists

9.1 Pre-launch checklist (week before)

Day -7 (Layer 1): - [ ] Wikidata entity created with all P-claims (§3.1) - [ ] llms.txt published at `/llms.txt` (§3.2) - [ ] JSON-LD Organization + WebSite + WebApplication on every page (§3.3)

Day -5 (Layer 2): - [ ] OpenAPI spec at `/api/v1/openapi.json` with examples on every endpoint (§4.2) - [ ] Sitemap at `/sitemap.xml`, robots at `/robots.txt` - [ ] FAQ schemas generated per page type (§4.5)

Day -3 (Layer 3): - [ ] Security headers configured (CSP, HSTS, X-Frame-Options, Permissions-Policy) - [ ] securityheaders.com score = A+ - [ ] RLS enabled on every table with explicit policies (§5.3) - [ ] All env vars set with `printf %s`, not `echo` (§5.4) - [ ] All third-party API calls wrapped in try/catch with reason-surfacing (§6.5)

Day -1 (Layer 4): - [ ] GA4 events fire correctly (verify via DevTools `window.dataLayer` inspect) - [ ] OG cards render properly on every page type (verify with platform debuggers) - [ ] Stripe success_url includes tier param (§6.2) - [ ] Test the full signup → checkout → success flow in incognito with a fresh email - [ ] All recent deployments show "Ready" status (§8.11) - [ ] mail-tester.com 10/10 (§0.7)

Launch day (Layer 5): - [ ] Hugging Face dataset README live (if you have a public dataset) - [ ] GitHub public docs repo published with topics set - [ ] IndexNow fired for every important URL - [ ] Social posts queued (X thread, LinkedIn, Bluesky) - [ ] Show HN submission scheduled for Tuesday 7-9 AM ET - [ ] 5 journalist pitches drafted; ready to send - [ ] Press release submitted to one wire service (optional but recommended)

9.2 Weekly health checks

Every Monday morning:

- [ ] Hosting dashboard — no failed builds in last 7 days
- [ ] GA4 Realtime — `purchase` events firing as expected
- [ ] Stripe webhook delivery rate (Stripe dashboard → Developers → Webhooks) — should be 100%
- [ ] Email deliverability (sender provider dashboard) — bounce rate < 2%, spam rate < 0.1%
- [ ] securityheaders.com still A+ on production URL
- [ ] Wikidata entity hasn't been vandalized (visit + spot-check)
- [ ] Sitemap reachable + up-to-date
- [ ] llms.txt reachable
- [ ] DMARC report email arrived for the previous week

9.3 Recovery playbook for common bugs

"**This page isn't working**" **on production:** 1. Check deploy status. Most recent must be "Ready". 2. If "Error", read build logs. Common cause: TypeScript type error you didn't catch locally. 3. If "Ready" but page broken, stream runtime logs. Reproduce the bug in your browser — runtime logs are real-time only.

Stripe SDK returns "Connection error": 1. Pull production env vars and inspect the Stripe vars 2. Look for trailing `\n` text or newline character in the values 3. Re-set with `printf %s "value" | <cli> env add` 4. Verify code path also does `.trim()` defensively

GA4 events not appearing in Realtime: 1. Open DevTools on the page that should fire the event 2. Check `window.dataLayer` after the action. Should contain `['event', 'name', {...}]` 3. If missing, your event code didn't execute. Check console for JS errors. 4. If present, it should appear in GA4 Realtime within 30s. If not after 5 min, check: - GA4 Admin → Property settings — Measurement ID matches your code - DebugView (separate from Realtime; uses different data path) - No ad blocker / privacy extension blocking requests to google-analytics.com

Slug URL 404s on production: 1. Verify the row exists in DB with the expected slug 2. Check for duplicate slugs
(`SELECT slug, count(*) FROM events GROUP BY slug HAVING count(*) > 1`) 3. If duplicates: append a uniqueness suffix (date, ID) to the slug, run a backfill SQL 4. Bust browser cache: change response code from 308 to 307 on the canonical-redirect route

Social previews still broken after OG fix: 1. Fetch your OG URL directly with curl — must return image data, status 200, content-type `image/png` 2. Use platform-specific re-scrape tools (X, Facebook, LinkedIn — links in §8.9) 3. If using edge-runtime OG generation, verify your edge runtime hasn't crashed

Magic-link emails not arriving: 1. Check mail-tester.com score on a freshly-sent message 2. Check Google Postmaster Tools for reputation degradation 3. Verify SPF / DKIM / DMARC records via your DNS console 4. Test from a different recipient (Gmail vs Outlook vs Yahoo) — pattern often reveals the failure mode

---

§10. Templates + appendix

See `/templates/` for ready-to-paste artifacts:

- `llms.txt` — fill-in-the-blanks template
- `webApplicationLd.ts` — JSON-LD WebApplication generator (typed)
- `csp-reference.md` — Reference CSP for SaaS with Stripe + GA4 + Supabase
- `journalist-pitch.md` — Outreach email template

- `wikidata-quickstatements.md` — QS batch script template

---

§11. Closing thoughts

The web is changing under our feet faster than at any point since mobile.

LLMs are the new search engines. Wikidata is the new Encyclopedia Britannica. JSON-LD is the new sitemap.xml. OpenAPI is the new API documentation page. Hugging Face is the new data partnerships page. Direct journalist outreach is the new SEO.

What hasn't changed: building a thing people want, telling people about it clearly, and getting out of their way. Layer 5 (distribution) without Layers 1-4 (foundation) is shouting in an empty room. Layers 1-4 without Layer 5 is a library nobody visits.

Build all five. The compounding is what works.

Kenny Hyder runs Hyder Media, a digital marketing consultancy. He's been doing performance marketing, conversion optimization, and analytics since 2009. Reach him at kenny@hyder.me or hyder.me.

This playbook is documented from real production deployments. Fork it, adapt it, distribute it. If you publish a derivative, attribution to Hyder Media (or this playbook URL) is appreciated but not required.
